Experiment 1

1.a. Analyze the trend of Data Science job postings over the last decade and visualize it using Pandas and Matplotlib.
1.b. Analyze and visualize the distribution of various Data Science roles from a dataset.
1.c. Differentiate Structured, Unstructured and Semi-structured data based on the given datasets.
1.d. Encrypt and decrypt sensitive data using the Cryptography library Fernet.

Code 1

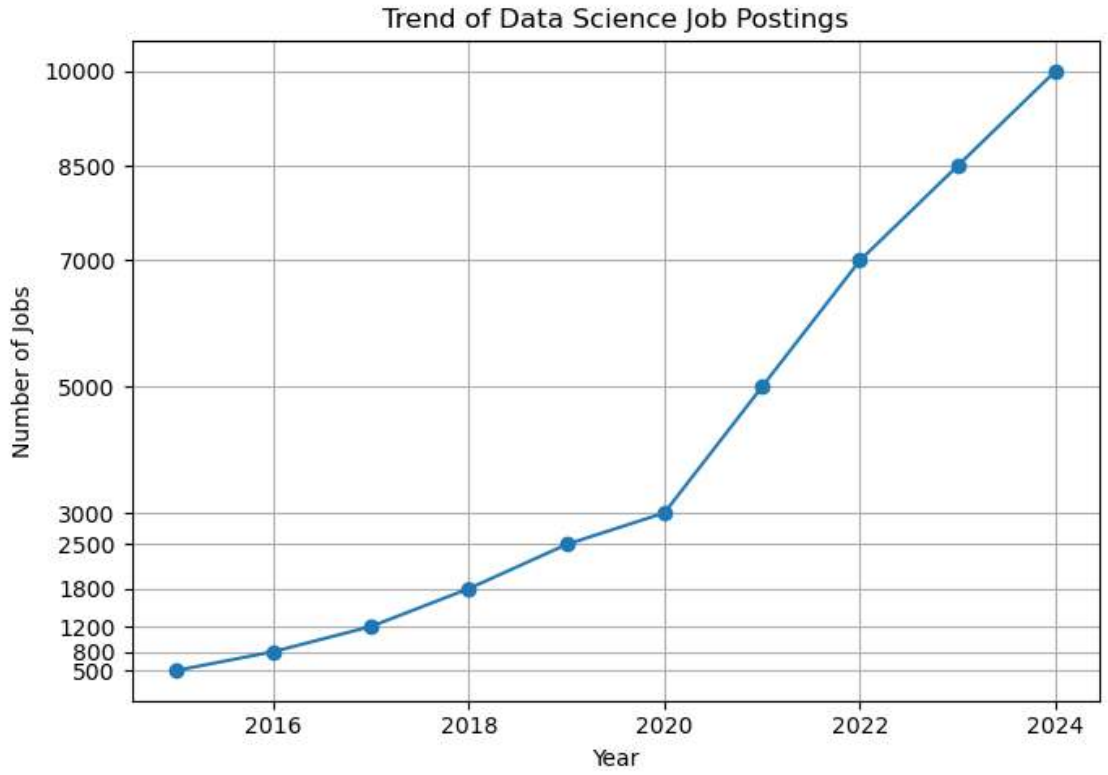
```
import pandas as pd
import matplotlib.pyplot as plt
years = [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]
jobs = [500, 800, 1200, 1800, 2500, 3000, 5000, 7000, 8500, 10000]
df_trend = pd.DataFrame({'Year': years, 'Number of Jobs': jobs})
print(df_trend)
plt.figure(figsize=(7, 5))
plt.plot(df_trend['Year'], df_trend['Number of Jobs'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Jobs')
plt.yticks(jobs)
plt.grid(True)
plt.tight_layout()
```

Output

```
   Year  Number of Jobs
0  2015             500
1  2016             800
2  2017            1200
3  2018            1800
4  2019            2500
5  2020            3000
6  2021            5000
7  2022            7000
8  2023            8500
9  2024           10000
```

Output

```
<Figure size 700x500 with 1 Axes>
```



Code 2

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
roles = ['Data Scientist', 'Data Scientist', 'Data Scientist',
 'Data Analyst', 'Data Analyst',
 'Data Engineer', 'Data Engineer',
 'ML Engineer', 'ML Engineer']
df_roles = pd.DataFrame({'Role': roles})
role_counts = df_roles['Role'].value_counts().reindex(
 ['Data Scientist', 'Data Analyst', 'Data Engineer', 'ML Engineer'])
print(role_counts)
plt.figure(figsize=(6, 5))
role_counts.plot(kind='bar')
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.tight_layout()
```

Output

```
Role
Data Scientist    3
Data Analyst      2
Data Engineer     2
ML Engineer       2
Name: count, dtype: int64
```

Output

```
<Figure size 600x500 with 1 Axes>
```



Code 3

```
import pandas as pd
import json

structured_data = {
 'StudentID': [101, 102, 103],
 'Name': ['Arjun', 'Priya', 'Karthik'],
 'Marks': [88, 92, 79]
}
df_structured = pd.DataFrame(structured_data)
df_structured

semi_structured_data = '''
[
 {"StudentID": 101, "Name": "Arjun", "Marks": {"Math": 88, "Science": 91}},
 {"StudentID": 102, "Name": "Priya", "Subjects": ["Math", "English"], "Marks": 92}
]
'''
```

```
parsed = json.loads(semi_structured_data)
print(json.dumps(parsed, indent=2))

unstructured_data = """Student Arjun scored well in Math this term and showed
great improvement in Science."""
print(unstructured_data)
```

Output

```
[
  {
    "StudentID": 101,
    "Name": "Arjun",
    "Marks": {
      "Math": 88,
      "Science": 91
    }
  },
  {
    "StudentID": 102,
    "Name": "Priya",
    "Subjects": [
      "Math",
      "English"
    ],
    "Marks": 92
  }
]
Student Arjun scored well in Math this term and showed
great improvement in Science.
```

Code 4

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)
print("Generated Key:", key.decode())

sensitive_data = "Student Name: Arjun, Aadhar No: 1234-5678-9012, Marks: 88"
print("\nOriginal Data:", sensitive_data)

data_bytes = sensitive_data.encode()
encrypted_data = cipher.encrypt(data_bytes)
print("\nEncrypted Data:", encrypted_data.decode())

decrypted_data = cipher.decrypt(encrypted_data)
print("\nDecrypted Data:", decrypted_data.decode())

print("\nMatch Original? ", decrypted_data.decode() == sensitive_data)
```

Output

```
Generated Key: L48ZIAoAwJvTYIrJ3LvQY1qrMiWIFasVRUb8C1MfbGk=

Original Data: Student Name: Arjun, Aadhar No: 1234-5678-9012, Marks: 88

Encrypted Data: gAAAAABqdHuocUOs9xi3zmdx2vlzBQ2P2sYg3i6f5ut9hDuMr0v961lWUV05HmOgL8pcsMYuWV4bh54Gu3PiXRSPAvf8oQgEFJ63Hd

Decrypted Data: Student Name: Arjun, Aadhar No: 1234-5678-9012, Marks: 88

Match Original?  True
```

Experiment 2

Upload and analyze a CSV dataset and perform data preprocessing and visualization.

Code 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

file_path = 'sales_data.csv'
df = pd.read_csv(file_path)

print(df.head())
print(df.isnull().sum())

df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)

print(df.describe())

product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

print(product_summary)

plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()

pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc=np.sum,
    fill_value=0
)
print(pivot_table)

correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

Output

```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
```

```
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```

Output

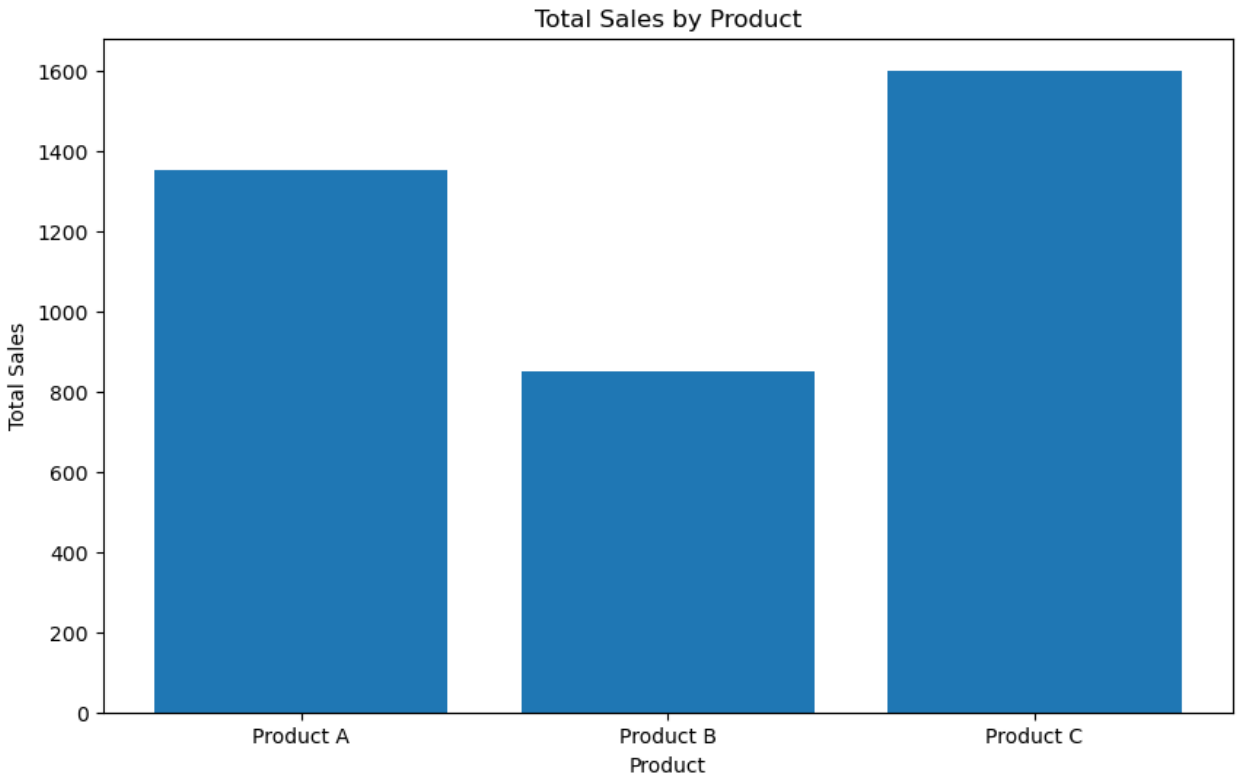
```
C:\Users\vsros\AppData\Local\Temp\ipykernel_16952\3353328291.py:12: FutureWarning: A value is trying to be set on a co
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which w

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df
```

```
df['Sales'].fillna(df['Sales'].mean(), inplace=True)
```

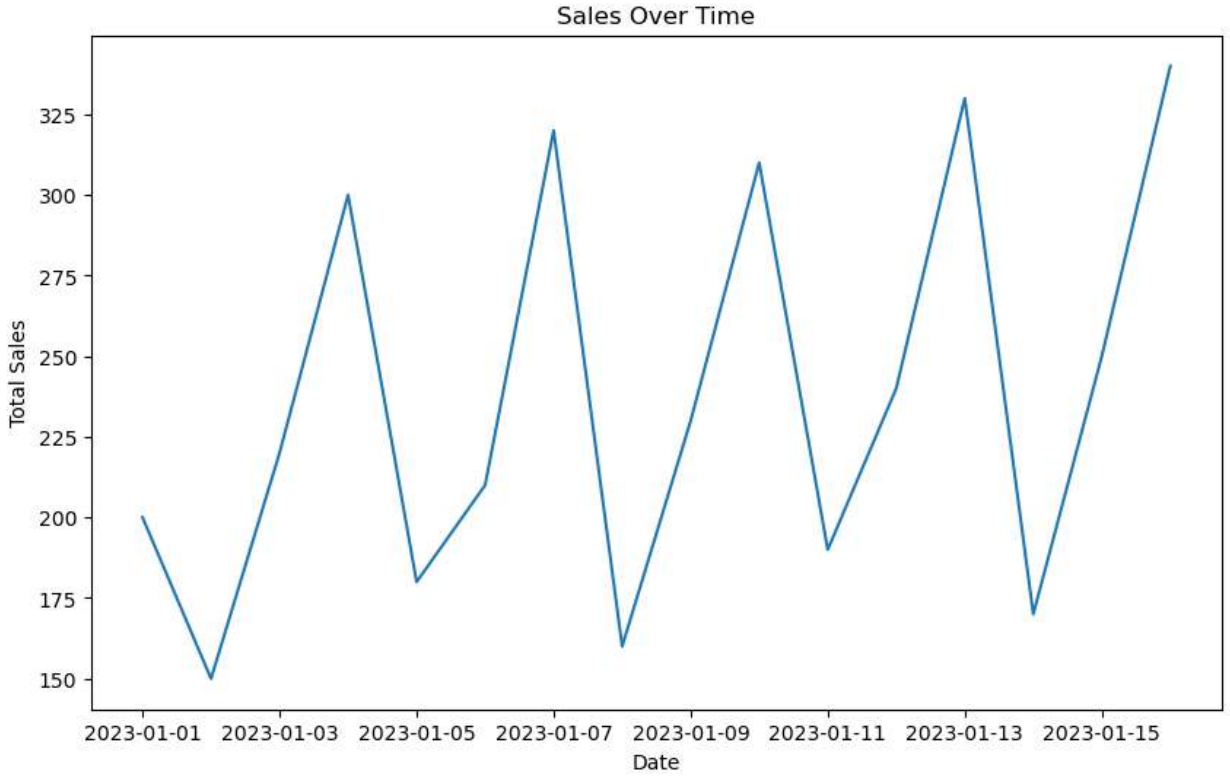
Output

```
<Figure size 1000x600 with 1 Axes>
```



Output

```
<Figure size 1000x600 with 1 Axes>
```



Output

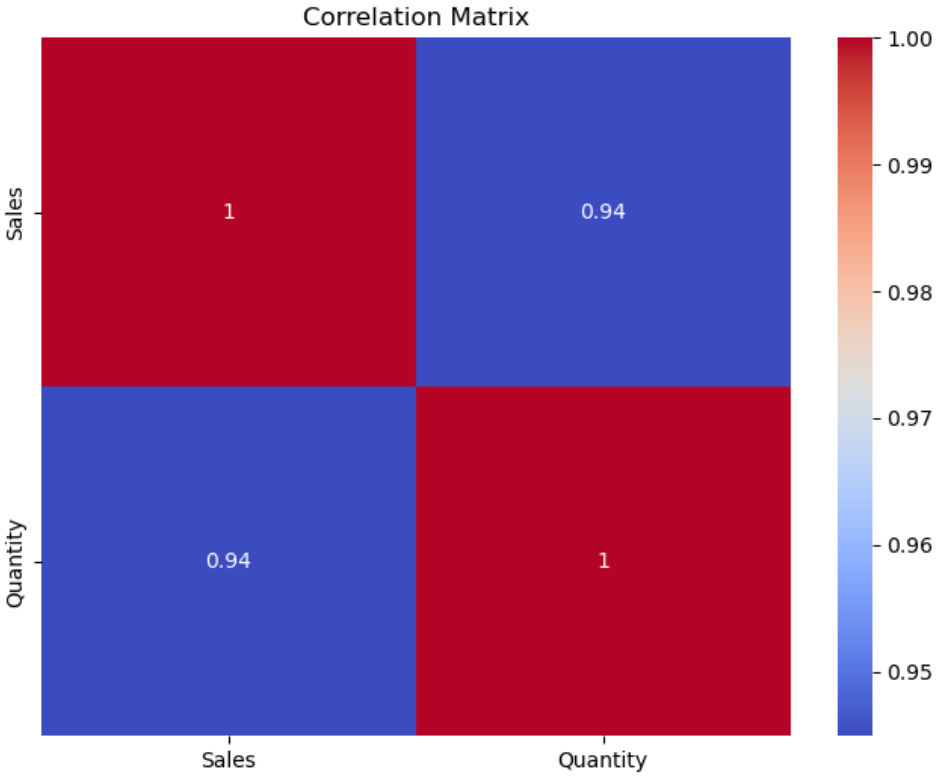
```
Product  Product A  Product B  Product C
Region
East             0          0       1600
North         1350          0          0
South            0        480          0
West             0        370          0
              Sales  Quantity
Sales      1.000000  0.944922
Quantity   0.944922  1.000000
```

Output

```
C:\Users\vsros\AppData\Local\Temp\ipykernel_16952\3353328291.py:41: FutureWarning: The provided callable <function sum
  pivot_table = df.pivot_table(
```

Output

```
<Figure size 800x600 with 2 Axes>
```


Correlation Matrix
Sales
Quantity
1
0.94
0.94
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

Experiment 3

Handling missing and inappropriate data in a dataset.

Code 1

```
import numpy as np
import pandas as pd

df = pd.read_csv("Hotel_Dataset.csv")
print("Original Dataset:")
print(df)
print()

print("Duplicate rows check:")
print(df.duplicated())
print()

print("Dataset Info:")
df.info()
print()

df.drop_duplicates(inplace=True)
print("Dataset after removing duplicates:")
print(df)
print()
print("Number of rows after removing duplicates:", len(df))
print()

index = np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
print("Dataset after resetting index:")
print(df)
print()

df.drop(['Age_Group.1'], axis=1, inplace=True)
print("Dataset after dropping redundant column:")
print(df)
print()

df['CustomerID'] = df['CustomerID'].mask(df['CustomerID'] < 0)
df['Bill'] = df['Bill'].mask(df['Bill'] < 0)
df['EstimatedSalary'] = df['EstimatedSalary'].mask(df['EstimatedSalary'] < 0)
df['NoOfPax'] = df['NoOfPax'].mask((df['NoOfPax'] < 1) | (df['NoOfPax'] > 20))
print("Dataset after converting inappropriate values to NaN:")
print(df)
print()

print("Unique Age_Group values:", df.Age_Group.unique())
print("Unique Hotel values:", df.Hotel.unique())
print("Unique FoodPreference values:", df.FoodPreference.unique())
print()

df['Hotel'] = df.Hotel.replace(['Ibys'], 'Ibis')
df['FoodPreference'] = df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg')
df['FoodPreference'] = df.FoodPreference.replace(['non-Veg'], 'Non-Veg')
print("Unique Hotel values after cleaning:", df.Hotel.unique())
print("Unique FoodPreference values after cleaning:", df.FoodPreference.unique())
print()




df['EstimatedSalary'] = df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()))
df['NoOfPax'] = df.NoOfPax.fillna(round(df.NoOfPax.median()))
df['Rating(1-5)'] = df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()))
df['Bill'] = df.Bill.fillna(round(df.Bill.mean()))
print("Final Cleaned Dataset:")
print(df)
```

Output

```
Original Dataset:
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
```

```
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
9            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2            40000       20-25
1         3            59000       30-35
2         2            30000       25-30
3         2           120000       20-25
4         2            45000         35+
5         2           122220         35+
6        -1            21122         35+
7       -10           345673       20-25
8         3           -99999       25-30
9         3           -99999       25-30
10        4            87777       30-35

Duplicate rows check:
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes

Dataset after removing duplicates:
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2            40000       20-25
1         3            59000       30-35
2         2            30000       25-30
3         2           120000       20-25
4         2            45000         35+
5         2           122220         35+
6        -1            21122         35+
7       -10           345673       20-25
8         3           -99999       25-30
10        4            87777       30-35

Number of rows after removing duplicates: 10

Dataset after resetting index:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
0           1     20-25            4       Ibis            veg  1300        2
1           2     30-35            5  LemonTree        Non-Veg  2000        3
2           3     25-30            6     RedFox            Veg  1322        2
```

```
3           4     20-25           -1  LemonTree            Veg  1234        2
4           5       35+            3       Ibis     Vegetarian   989        2
5           6       35+            3       Ibys        Non-Veg  1909        2
6           7       35+            4     RedFox     Vegetarian  1000       -1
7           8     20-25            7  LemonTree            Veg  2999      -10
8           9     25-30            2       Ibis        Non-Veg  3456        3
9          10     30-35            5     RedFox        non-Veg -6755        4

   EstimatedSalary Age_Group.1
0            40000       20-25
1            59000       30-35
2            30000       25-30
3           120000       20-25
4            45000         35+
5           122220         35+
6            21122         35+
7           345673       20-25
8           -99999       25-30
9            87777       30-35

Dataset after dropping redundant column:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  NoOfPax  \
0           1     20-25            4       Ibis            veg  1300        2
1           2     30-35            5  LemonTree        Non-Veg  2000        3
2           3     25-30            6     RedFox            Veg  1322        2
3           4     20-25           -1  LemonTree            Veg  1234        2
4           5       35+            3       Ibis     Vegetarian   989        2
5           6       35+            3       Ibys        Non-Veg  1909        2
6           7       35+            4     RedFox     Vegetarian  1000       -1
7           8     20-25            7  LemonTree            Veg  2999      -10
8           9     25-30            2       Ibis        Non-Veg  3456        3
9          10     30-35            5     RedFox        non-Veg -6755        4

   EstimatedSalary
0            40000
1            59000
2            30000
3           120000
4            45000
5           122220
6            21122
7           345673
8           -99999
9            87777

Dataset after converting inappropriate values to NaN:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25            4       Ibis            veg  1300.0
1           2     30-35            5  LemonTree        Non-Veg  2000.0
2           3     25-30            6     RedFox            Veg  1322.0
3           4     20-25           -1  LemonTree            Veg  1234.0
4           5       35+            3       Ibis     Vegetarian   989.0
5           6       35+            3       Ibys        Non-Veg  1909.0
6           7       35+            4     RedFox     Vegetarian  1000.0
7           8     20-25            7  LemonTree            Veg  2999.0
8           9     25-30            2       Ibis        Non-Veg  3456.0
9          10     30-35            5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0              NaN
9      4.0          87777.0

Unique Age_Group values: ['20-25' '30-35' '25-30' '35+']
Unique Hotel values: ['Ibis' 'LemonTree' 'RedFox' 'Ibys']
Unique FoodPreference values: ['veg' 'Non-Veg' 'Veg' 'Vegetarian' 'non-Veg']

Unique Hotel values after cleaning: ['Ibis' 'LemonTree' 'RedFox']
Unique FoodPreference values after cleaning: ['Veg' 'Non-Veg']

Final Cleaned Dataset:
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0           1     20-25            4       Ibis            Veg  1300.0
1           2     30-35            5  LemonTree        Non-Veg  2000.0
2           3     25-30            6     RedFox            Veg  1322.0
3           4     20-25           -1  LemonTree            Veg  1234.0
4           5       35+            3       Ibis            Veg   989.0
```

5	6	35+	3	Ibis	Non-Veg	1909.0
6	7	35+	4	RedFox	Veg	1000.0
7	8	20-25	7	LemonTree	Veg	2999.0
8	9	25-30	2	Ibis	Non-Veg	3456.0
9	10	30-35	5	RedFox	Non-Veg	1801.0

	NoOfPax	EstimatedSalary
0	2.0	40000.0
1	3.0	59000.0
2	2.0	30000.0
3	2.0	120000.0
4	2.0	45000.0
5	2.0	122220.0
6	2.0	21122.0
7	2.0	345673.0
8	3.0	96755.0
9	4.0	87777.0

Experiment 4

Experiment to understand data preprocessing in Data Science.

Code 1

```
import numpy as np
import pandas as pd

df = pd.read_csv("pre-process_datasample.csv")
print("Original Dataset:")
print(df)
print()

print("Dataset Info:")
df.info()
print()

print("Mode of Country column:")
print(df.Country.mode())
print()




df['Country'] = df.Country.fillna(df.Country.mode()[0])
df['Age'] = df.Age.fillna(df.Age.median())
df['Salary'] = df.Salary.fillna(round(df.Salary.mean()))
print("Dataset after handling missing values:")
print(df)
print()

print("One-Hot Encoded Country column:")
print(pd.get_dummies(df.Country))
print()

updated_dataset = pd.concat([pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
axis=1)
print("Updated dataset after One-Hot Encoding:")
print(updated_dataset)
print()
print("Updated Dataset Info:")
updated_dataset.info()
print()

updated_dataset['Purchased'] = updated_dataset.Purchased.replace(['No', 'Yes'], [0,
1])
print("Final Preprocessed Dataset:")
print(updated_dataset)
```

Output

```
Original Dataset:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes

Mode of Country column:
0    France
```

```
Name: Country, dtype: object

Dataset after handling missing values:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes

One-Hot Encoded Country column:
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8   False     True  False
9    True    False  False

Updated dataset after One-Hot Encoding:
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
8   False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0       Yes

Updated Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   France     10 non-null     bool
 1   Germany    10 non-null     bool
 2   Spain      10 non-null     bool
 3   Age        10 non-null     float64
 4   Salary     10 non-null     float64
 5   Purchased  10 non-null     object
dtypes: bool(3), float64(2), object(1)
memory usage: 402.0+ bytes

Final Preprocessed Dataset:
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8   False     True  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1
```

Output

```
C:\Users\vsros\AppData\Local\Temp\ipykernel_20308\1860359933.py:40: FutureWarning: Downcasting behavior in `replace` is
  updated_dataset['Purchased'] = updated_dataset.Purchased.replace(['No', 'Yes'], [0,
```

Experiment 5

Outliers Detection.

Code 1

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
np.random.seed(42)

array = np.random.randint(1, 100, 16)
print("Generated array:")
print(array)
print()
print("Mean:", array.mean())
print()

print("25th percentile:", np.percentile(array, 25))
print("50th percentile:", np.percentile(array, 50))
print("75th percentile:", np.percentile(array, 75))
print("100th percentile:", np.percentile(array, 100))
print()

def outDetection(array):
 Q1, Q3 = np.percentile(array, [25, 75])
 IQR = Q3 - Q1
 lr = Q1 - (1.5 * IQR)
 ur = Q3 + (1.5 * IQR)
 return lr, ur
lr, ur = outDetection(array)
print("Lower Range, Upper Range:", (lr, ur))
print()

plt.figure(figsize=(5, 4))
sns.histplot(array, kde=True)
plt.title("Distribution Before Outlier Removal")
plt.tight_layout()
plt.show()

new_array = array[(array > lr) & (array < ur)]
print("Array after removing outliers:")
print(new_array)
print()

plt.figure(figsize=(5, 4))
sns.histplot(new_array, kde=True)
plt.title("Distribution After Outlier Removal")
plt.tight_layout()
plt.show()
print("Number of outliers removed:", len(array) - len(new_array))
```

Output

```
Generated array:
[52 93 15 72 61 21 83 87 75 75 88 24  3 22 53  2]

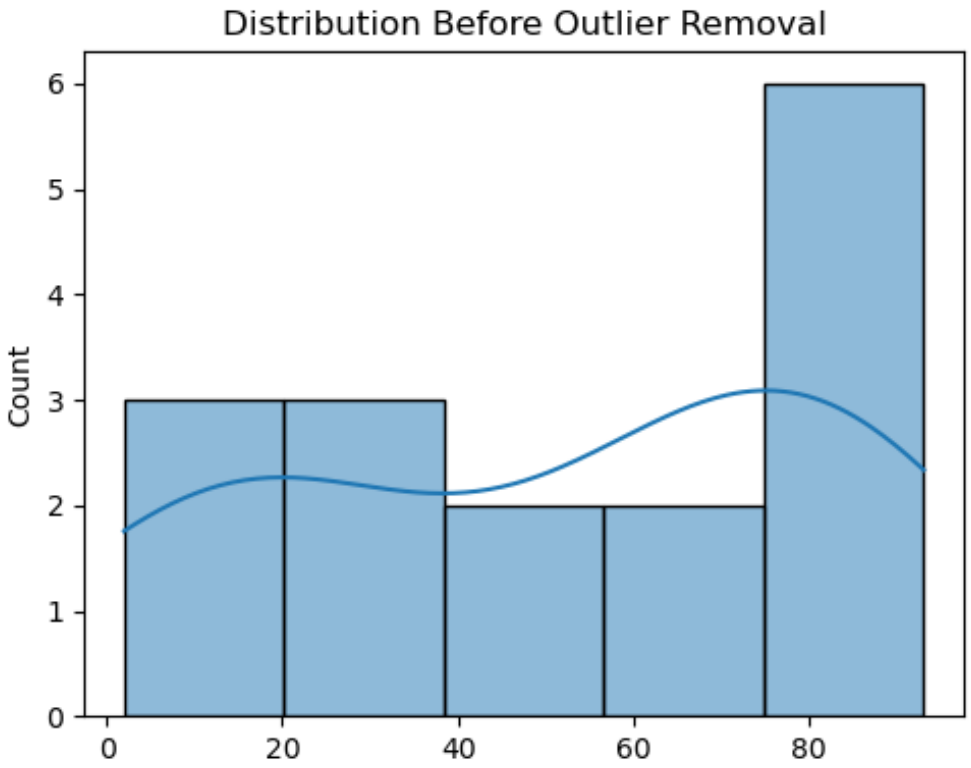
Mean: 51.625

25th percentile: 21.75
50th percentile: 57.0
75th percentile: 77.0
100th percentile: 93.0

Lower Range, Upper Range: (np.float64(-61.125), np.float64(159.875))
```

Output

```
<Figure size 500x400 with 1 Axes>
```

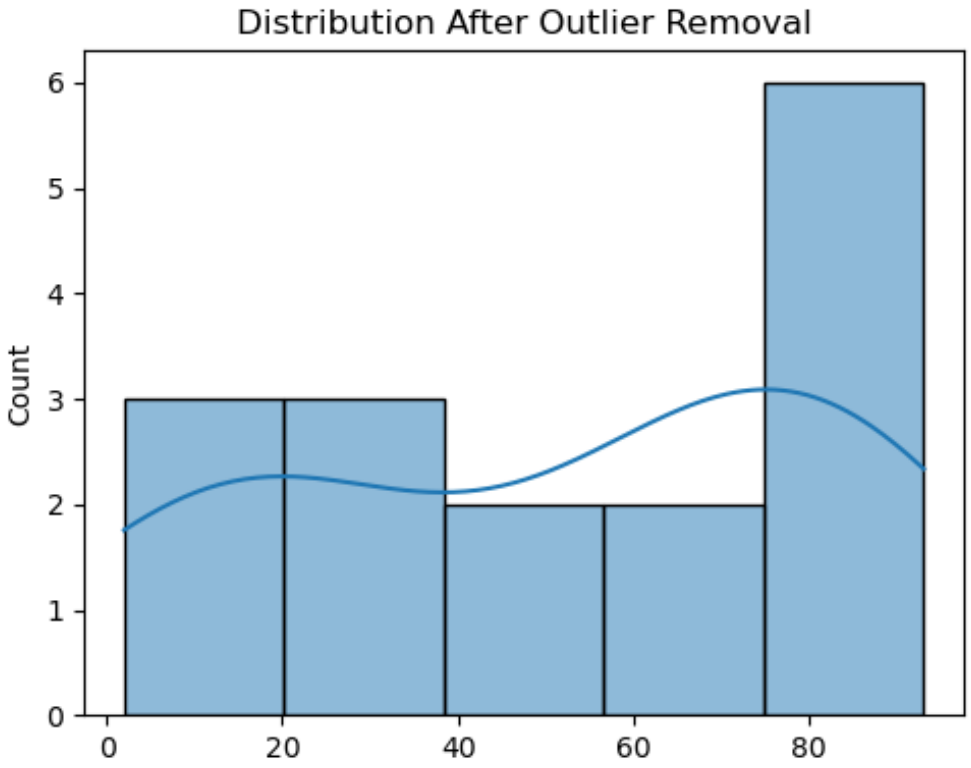


Output

```
Array after removing outliers:
[52 93 15 72 61 21 83 87 75 75 88 24  3 22 53  2]
```

Output

```
<Figure size 500x400 with 1 Axes>
```



Output

```
Number of outliers removed: 0
```

Experiment 6: Feature Scaling

Code

```
import numpy as np
import pandas as pd

df = pd.read_csv('pre-process_datasample.csv')
print("Original Dataset:")
print(df)
print()

df['Country'] = df.Country.fillna(df.Country.mode()[0])

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

from sklearn.impute import SimpleImputer
age_imputer = SimpleImputer(strategy="mean", missing_values=np.nan)
salary_imputer = SimpleImputer(strategy="mean", missing_values=np.nan)
age_imputer.fit(features[:, [1]])
salary_imputer.fit(features[:, [2]])
features[:, [1]] = age_imputer.transform(features[:, [1]])
features[:, [2]] = salary_imputer.transform(features[:, [2]])
print("Features after handling missing values:")
print(features)
print()

from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
print("One-Hot Encoded Country:")
print(Country)
print()

final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
print("Final feature set (before scaling):")
print(final_set)
print()

from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)
print("Feature set after Standard Scaling:")
print(feat_standard_scaler)
print()

from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)
print("Feature set after Min-Max Scaling:")
print(feat_minmax_scaler)
```

Output

```
Original Dataset:
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes

Features after handling missing values:
[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 63777.77777777778]
 ['France' 35.0 58000.0]
 ['Spain' 38.77777777777778 52000.0]
```

```
 ['France' 48.0 79000.0]
 ['Germany' 50.0 83000.0]
 ['France' 37.0 67000.0]]

One-Hot Encoded Country:
[[1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]]

Final feature set (before scaling):
[[1.0 0.0 0.0 44.0 72000.0]
 [0.0 0.0 1.0 27.0 48000.0]
 [0.0 1.0 0.0 30.0 54000.0]
 [0.0 0.0 1.0 38.0 61000.0]
 [0.0 1.0 0.0 40.0 63777.77777777778]
 [1.0 0.0 0.0 35.0 58000.0]
 [0.0 0.0 1.0 38.77777777777778 52000.0]
 [1.0 0.0 0.0 48.0 79000.0]
 [0.0 1.0 0.0 50.0 83000.0]
 [1.0 0.0 0.0 37.0 67000.0]]

Feature set after Standard Scaling:
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00  0.00000000e+00
  -1.07356980e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  1.34013983e+00
   1.38753832e+00]
 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.63077256e+00
   1.75214693e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -2.58340208e-01
   2.93712492e-01]]

Feature set after Min-Max Scaling:
[[1.         0.         0.         0.73913043 0.68571429]
 [0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.13043478 0.17142857]
 [0.         0.         1.         0.47826087 0.37142857]
 [0.         1.         0.         0.56521739 0.45079365]
 [1.         0.         0.         0.34782609 0.28571429]
 [0.         0.         1.         0.51207729 0.11428571]
 [1.         0.         0.         0.91304348 0.88571429]
 [0.         1.         0.         1.         1.        ]
 [1.         0.         0.         0.43478261 0.54285714]]
```

Experiment 7

EDA – Quantitative and Qualitative Analysis.

Program:

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
tips = sns.load_dataset('tips')
print(tips.head())
sns.displot(tips.total_bill, kde=True)
sns.displot(tips.total_bill, kde=False)
sns.jointplot(x=tips.tip, y=tips.total_bill)
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
sns.pairplot(tips)
print(tips.time.value_counts())
sns.pairplot(tips, hue='time')
sns.pairplot(tips, hue='day')
sns.heatmap(tips.corr(numeric_only=True), annot=True)
sns.boxplot(x=tips.total_bill)
sns.boxplot(x=tips.tip)
sns.countplot(x=tips.day)
sns.countplot(x=tips.sex)
tips.sex.value_counts().plot(kind='pie')
tips.sex.value_counts().plot(kind='bar')
sns.countplot(x=tips[tips.time=='Dinner']['day'])
print("All plots generated successfully.")
```

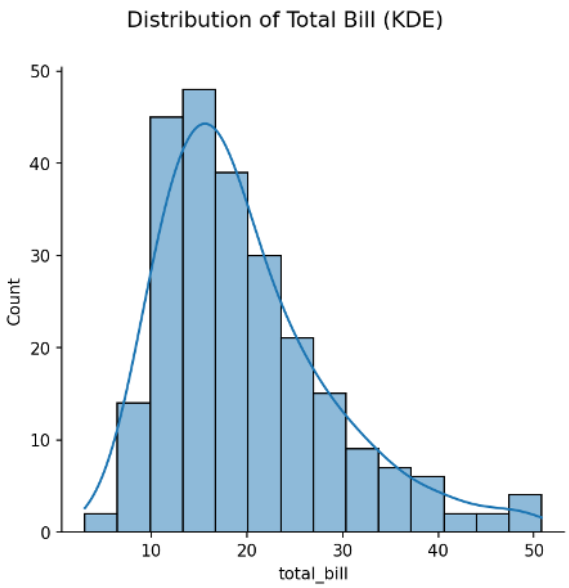
Output

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

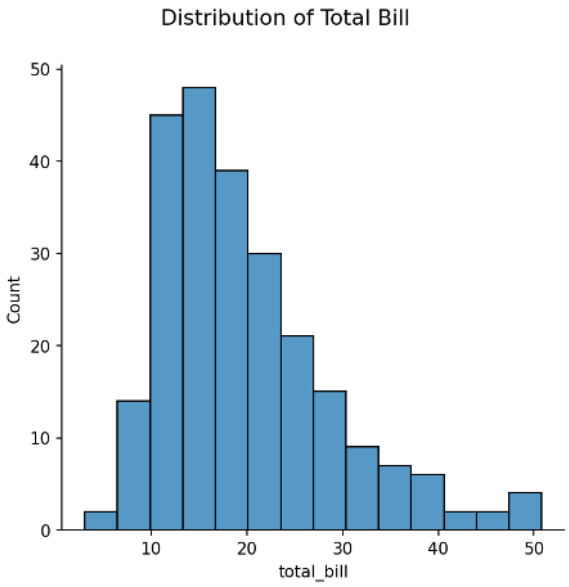
```
time
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

```
All plots generated successfully.
```

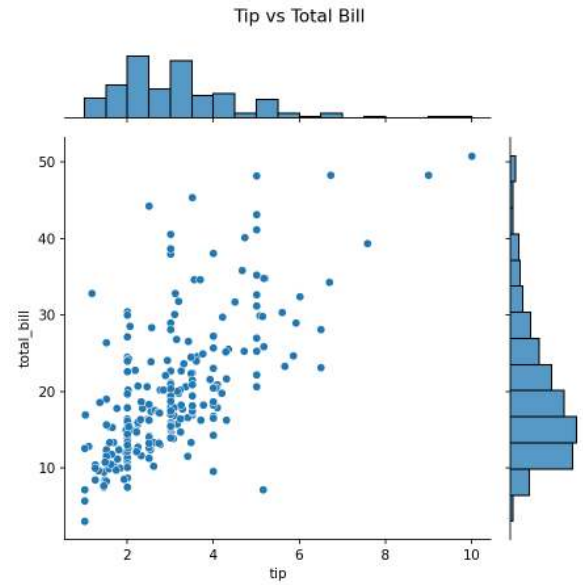
Output



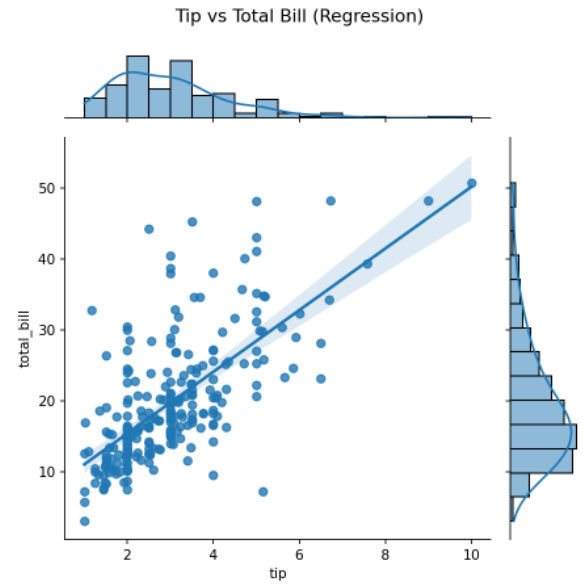
Output



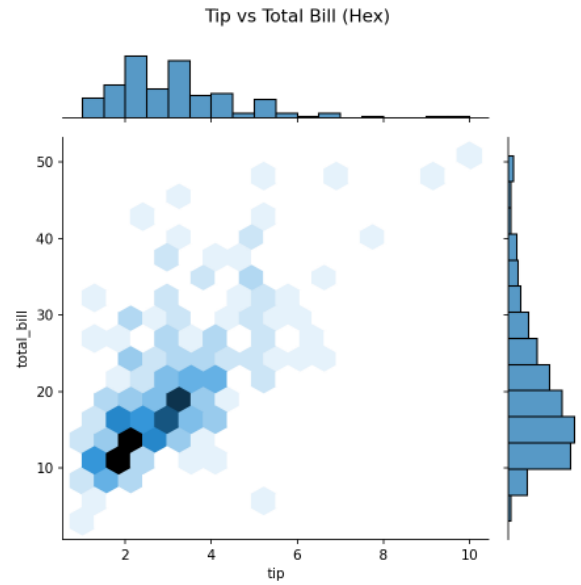
Output



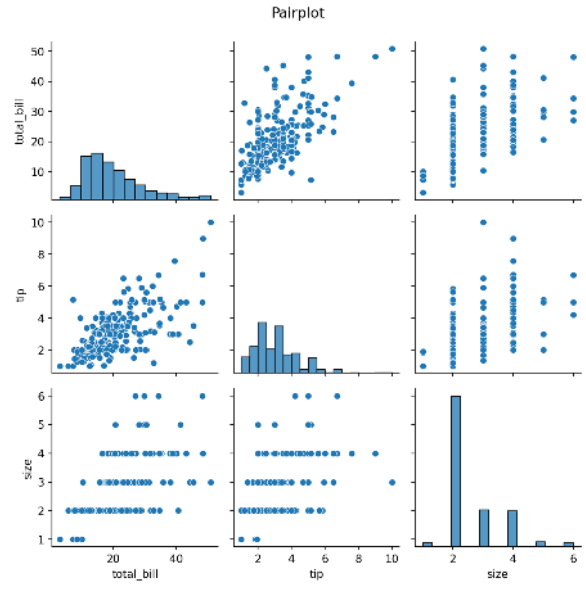
Output



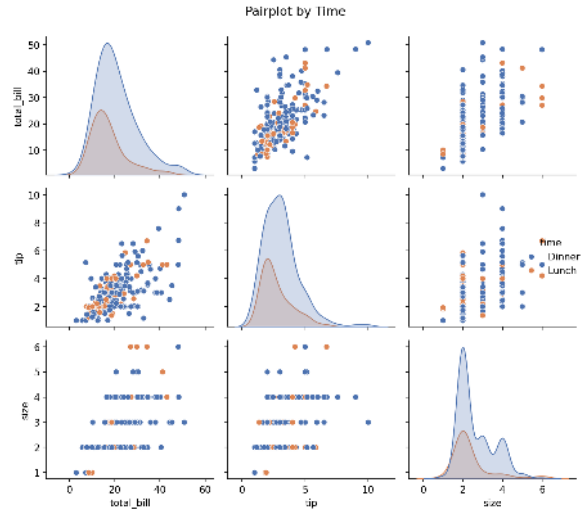
Output



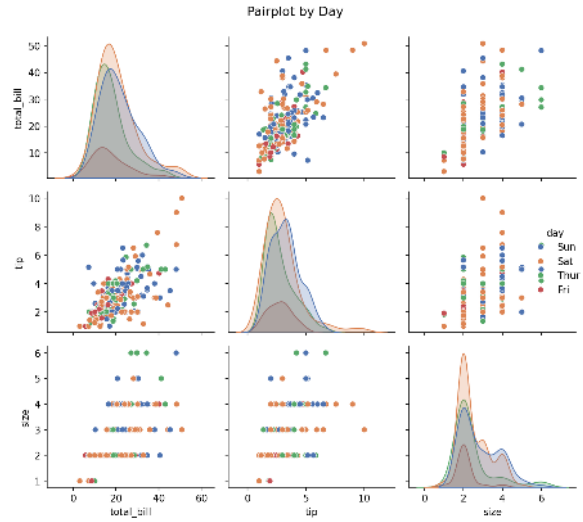
Output



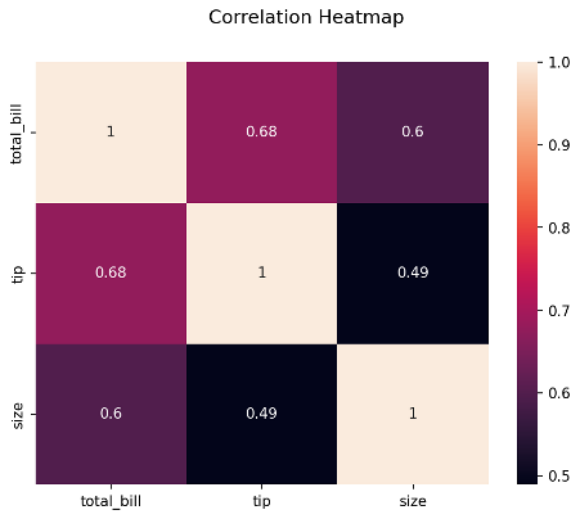
Output



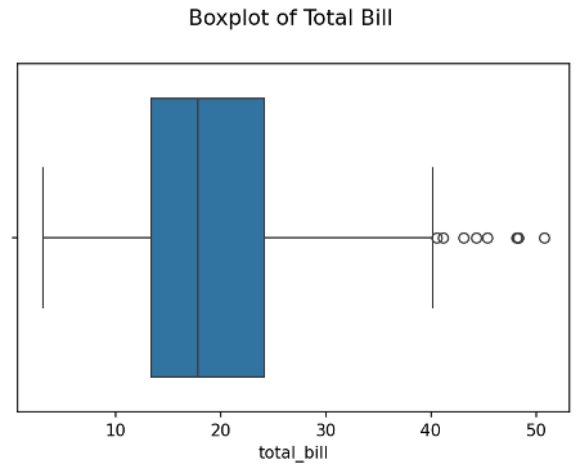
Output



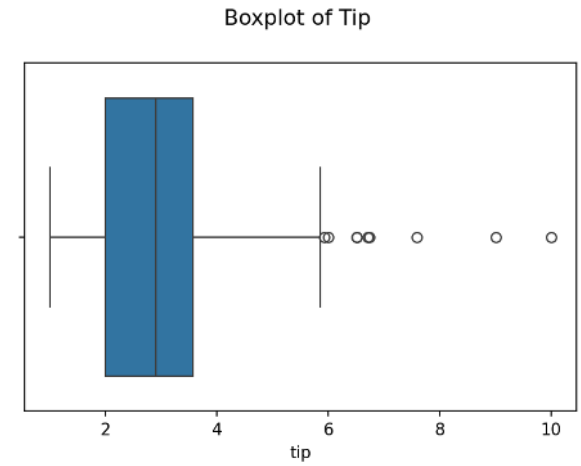
Output



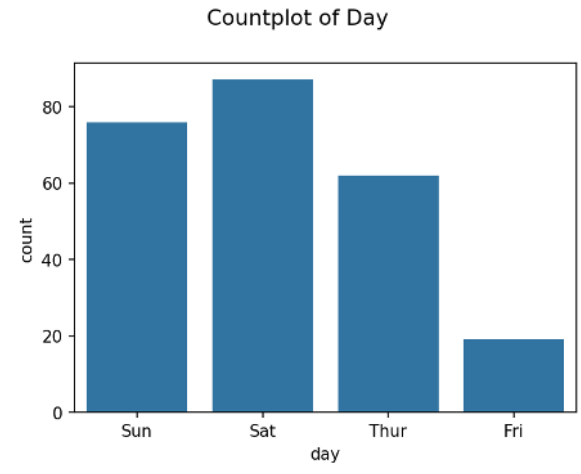
Output



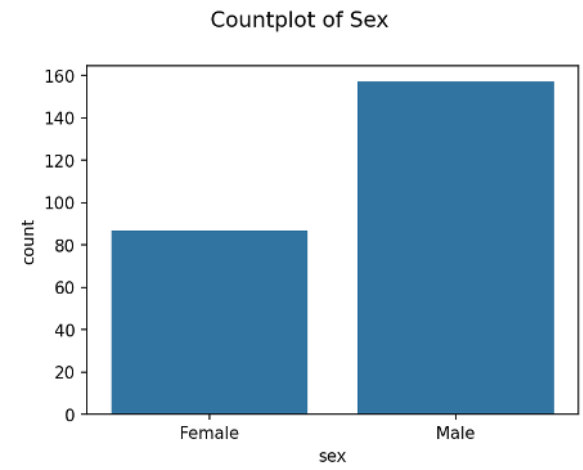
Output



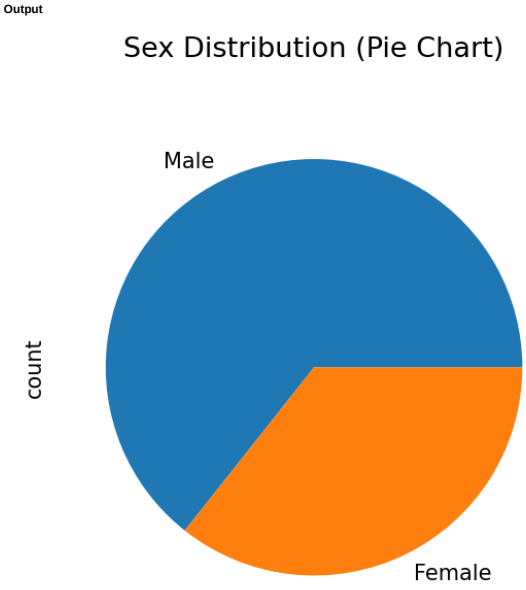
Output



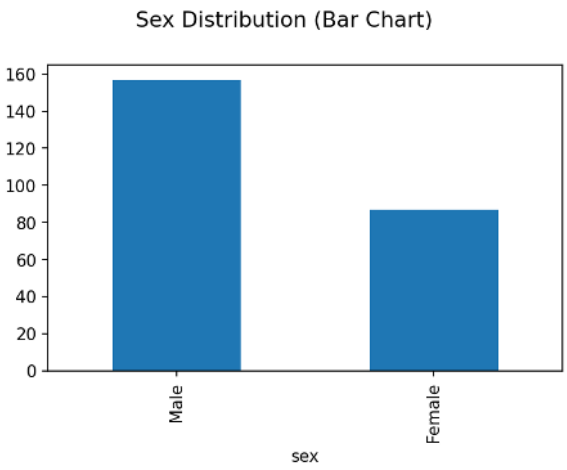
Output



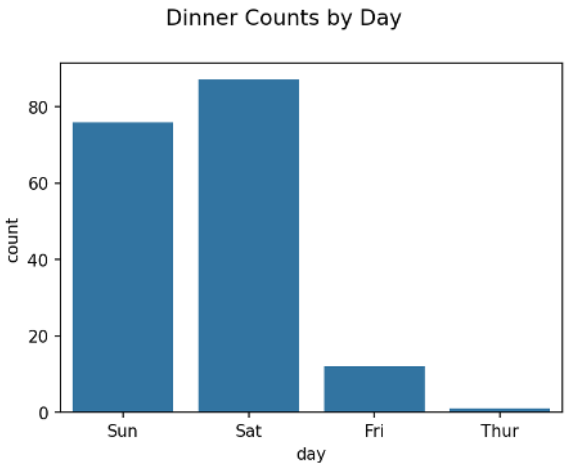
Output



Output



Output



Experiment 8

Linear Regression.

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
df.info()
df.dropna(inplace=True)
df.info()
df.describe()
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,
test_size=0.2,random_state=23)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test)
model.coef_
model.intercept_
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}: "
.format(yr_of_exp,Salary))
```

Output

```
   YearsExperience  Salary
0              1.1   39343
1              1.3   46205
2              1.5   37731
3              2.0   43525
4              2.2   39891

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   YearsExperience 30 non-null     float64
 1   Salary          30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

Output

```
       YearsExperience         Salary
count       30.000000      30.000000
mean         5.313333   76003.000000
std          2.837888   27414.429785
min          1.100000   37731.000000
25%          3.200000   56720.750000
50%          4.700000   65237.000000
75%          7.700000  100544.750000
max         10.500000  122391.000000

LinearRegression()

0.960318254743891

0.918417084921423

array([[9281.30847068078]])

array([27166.73682891278])
```

Output

```
Enter Years of Experience: 44
Estimated Salary for 44.0 years of experience is [[435544.30953887]]:
```

Experiment 9

Logistic Regression.

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df
df.head()

features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

for i in range(1,401):
    x_train,x_test,y_train,y_test=train_test_split(
        features,label,test_size=0.2,random_state=i)
    model=LogisticRegression()
    model.fit(x_train,y_train)
    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)
    if test_score>train_score:
        print("Test {} Train{} Random State {}".format(
            test_score,train_score,i))

x_train,x_test,y_train,y_test=train_test_split(
    features,label,test_size=0.2,random_state=37)

finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)

print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))

from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

Output

```
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0

400 rows × 5 columns
```

```
   User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510  Male     19  19000             0
1  15810944  Male     35  20000             0
2  15668575  Female   26  43000             0
3  15603246  Female   27  57000             0
4  15804002  Male     19  76000             0
```

Output

```
Test 0.6875 Train0.63125 Random State 3
Test 0.7375 Train0.61875 Random State 4
Test 0.6625 Train0.6375 Random State 5
Test 0.65 Train0.640625 Random State 6
Test 0.675 Train0.634375 Random State 7
Test 0.675 Train0.634375 Random State 8
Test 0.65 Train0.640625 Random State 10
Test 0.6625 Train0.6375 Random State 11
Test 0.7125 Train0.625 Random State 13
Test 0.675 Train0.634375 Random State 16
Test 0.7 Train0.628125 Random State 17
Test 0.7 Train0.628125 Random State 21
Test 0.65 Train0.640625 Random State 24
Test 0.6625 Train0.6375 Random State 25
Test 0.75 Train0.615625 Random State 26
Test 0.675 Train0.634375 Random State 27
Test 0.7 Train0.628125 Random State 28
Test 0.6875 Train0.63125 Random State 29
Test 0.6875 Train0.63125 Random State 31
Test 0.6625 Train0.6375 Random State 37

LogisticRegression()

0.865625

0.8125

              precision    recall  f1-score   support

           0       0.85      0.93      0.89       257
           1       0.84      0.71      0.77       143

    accuracy                           0.85       400
   macro avg       0.85      0.82      0.83       400
weighted avg       0.85      0.85      0.85       400
```

Experiment 10

K-Means Clustering.

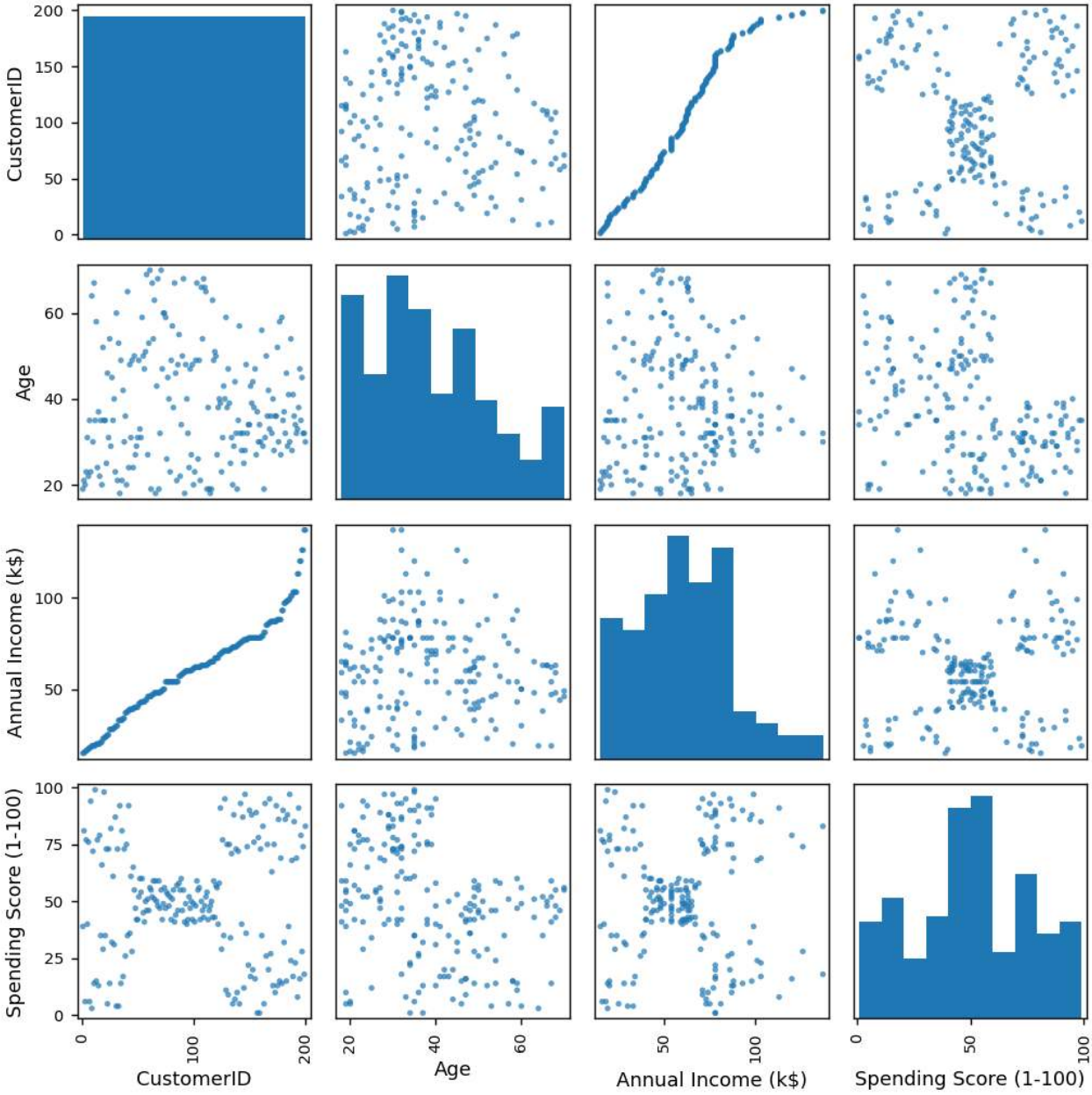
Program:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv('Mall_Customers.csv')
df.info()
df.head()
sns.pairplot(df)
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend()
plt.show()
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
    model=KMeans(n_clusters=i)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

Output

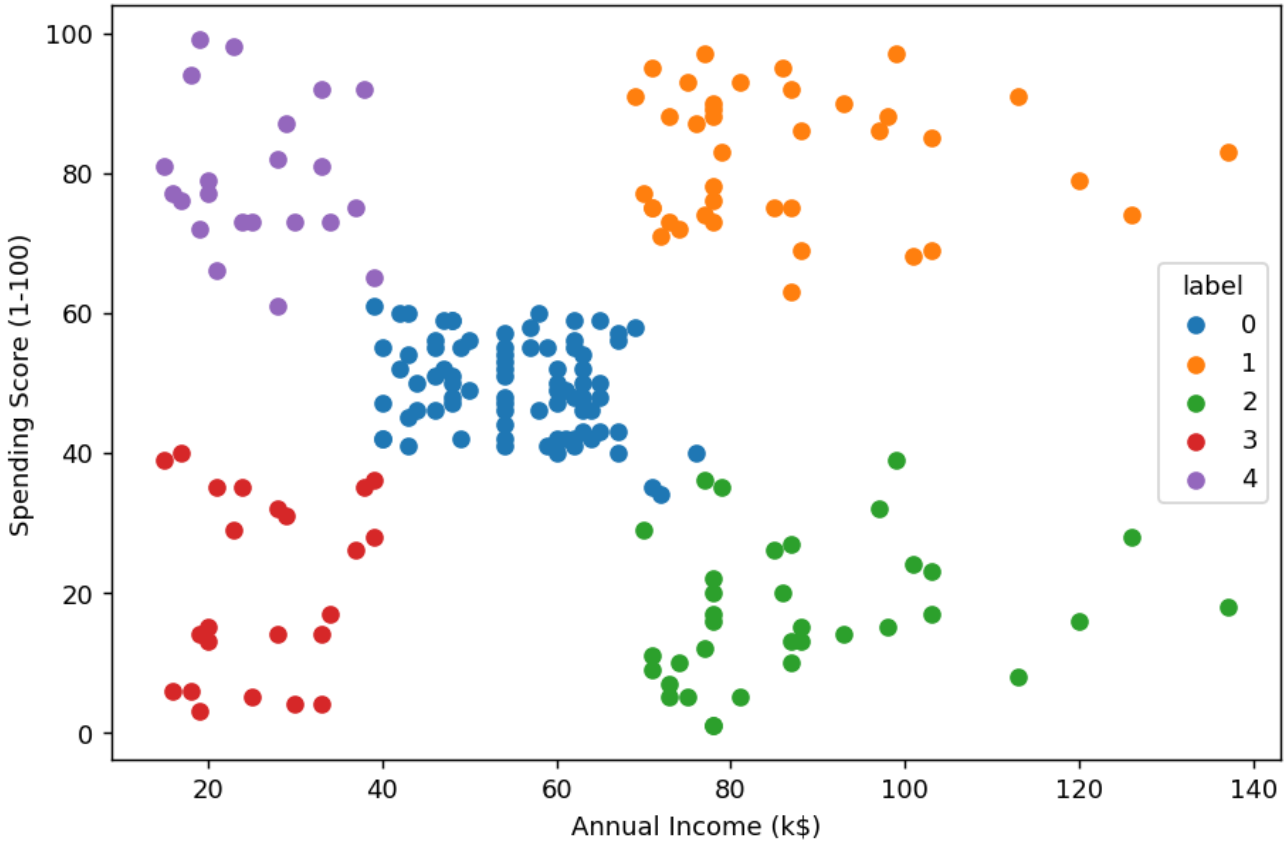
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
0   CustomerID              200 non-null    int64
1   Gender                  200 non-null    object
2   Age                     200 non-null    int64
3   Annual Income (k$)      200 non-null    int64
4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

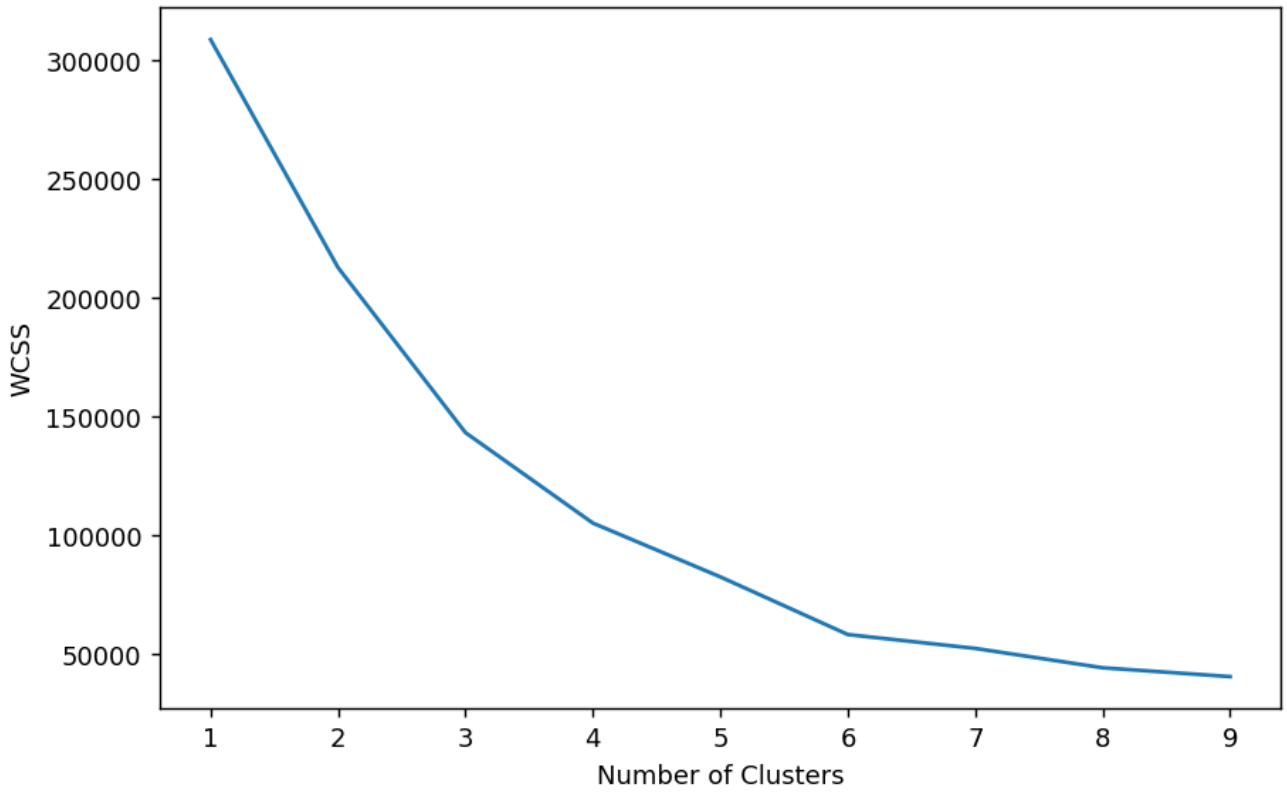
	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40


CustomerID
Age
Annual Income (k$)
Spending Score (1-100)
CustomerID
Age
Annual Income (k$)
Spending Score (1-100)

Output

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	3
1	15	81	4
2	16	6	3
3	16	77	4
4	17	40	3





Experiment 11

K-Nearest Neighbors (KNN).

Program:

```
import numpy as np
import pandas as pd
df=pd.read_csv('Iris.csv')
df.info()
df.variety.value_counts()
df.head()

features=df.iloc[:,:-1].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(
features,label,test_size=.2,random_state=1)

model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)

print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

Output

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
0   sepal.length   150 non-null    float64
1   sepal.width    150 non-null    float64
2   petal.length   150 non-null    float64
3   petal.width    150 non-null    float64
4   variety        150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

variety
Setosa        50
Versicolor    50
Virginica     50

   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa
```

Scores:

```
0.9583333333333334
1.0
```

Confusion Matrix:

```
[[50  0  0]
 [ 0 47  3]
 [ 0  2 48]]
```

Classification Report:

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
   Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

Experiment 12

Z Test, T Test, F Test and ANOVA.

Program:

T-test

```
import numpy as np
from scipy import stats

# Sample data
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])

# Hypothesized mean
mu_0 = 70

# One-sample t-test
t_stat, p_value = stats.ttest_1samp(marks, mu_0)

print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 70.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Z-test

```
import numpy as np
from math import sqrt
from scipy.stats import norm

# Given data
x_bar = 51.2 # sample mean
mu_0 = 50 # population mean
sigma = 3 # population standard deviation
n = 36 # sample size

# Calculate Z-statistic
z_stat = (x_bar - mu_0) / (sigma / sqrt(n))

# Two-tailed p-value
p_value = 2 * (1 - norm.cdf(abs(z_stat)))

print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Anova Test

```
import numpy as np
from scipy import stats

# Data
A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(A, B, C)

print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject Null Hypothesis → Means are significantly different.")
else:
    print("Fail to Reject Null Hypothesis → No significant difference.")
```

Output

```
T-test Output:
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.

Z-test Output:
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.

ANOVA / F-test Output:
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis → Means are significantly different.
```

Experiment 13(a)

Data Visualization using Line Plot.

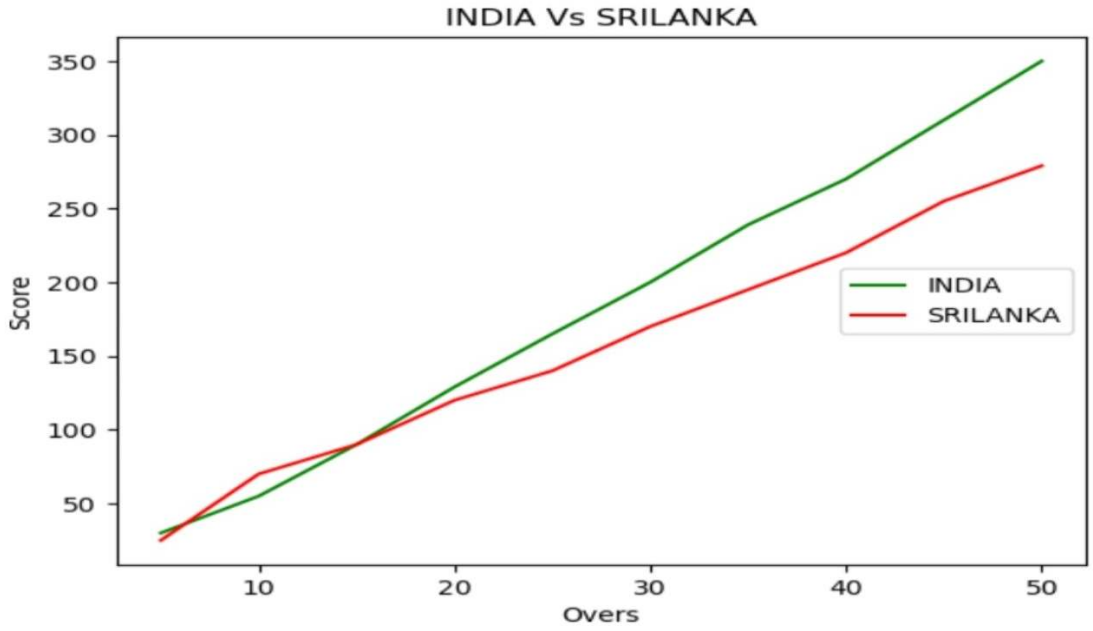
Program:

```
import matplotlib.pyplot as cricket

Overs = list(range(5, 51, 5))
Indian_Score = [30, 55, 90, 129, 165, 200, 239, 270, 310, 350]
Srilankan_Score = [25, 70, 90, 120, 140, 170, 195, 220, 255, 279]

cricket.plot(Overs, Indian_Score, color="green", label="INDIA")
cricket.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend(loc="center right")
cricket.show()
```

Output



Experiment 13(b)

Data Visualization using Bar Chart.

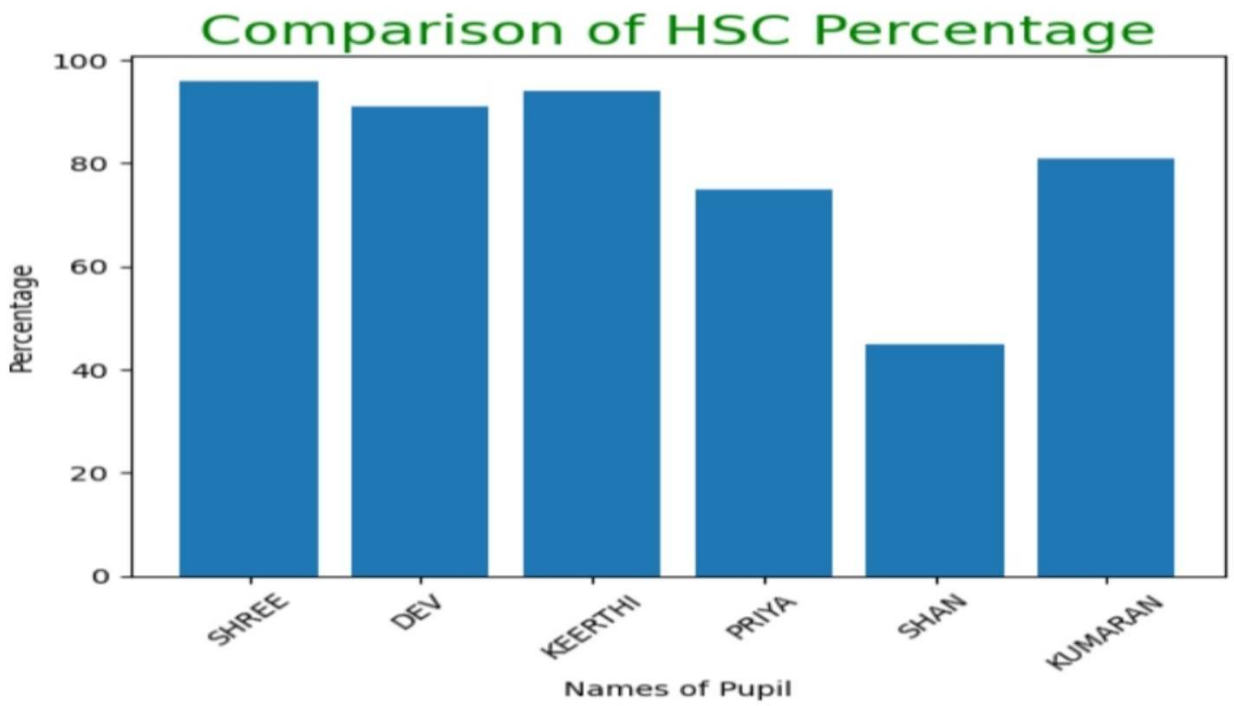
Program:

```
import matplotlib.pyplot as hscmark
import numpy as np

Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]

hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```

Output



Experiment 13(c)

Data Visualization using Pie Chart.

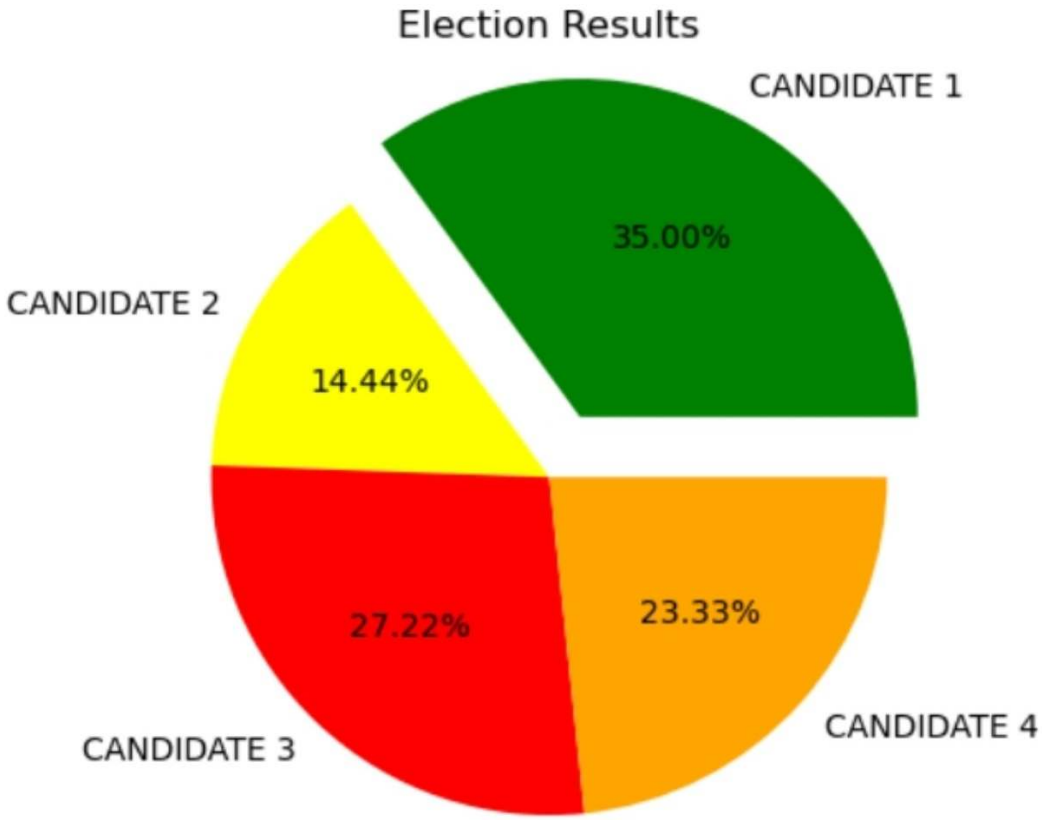
Program:

```
import matplotlib.pyplot as election

labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)

election.pie(Votes, labels=labels, colors=colors, explode=explode,
             autopct='%0.2f%%')
election.title('Election Results')
election.show()
```

Output



Experiment 14

Count the Frequency of Occurrence of a Word in a Body of Text.

Program:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

nltk.download("gutenberg")
nltk.download("punkt")
nltk.download("punkt_tab")

sample = gutenberg.raw("austen-emma.txt")
token = word_tokenize(sample)

wlist = []
for i in range(50):
    wlist.append(token[i])

wordfreq = [wlist.count(w) for w in wlist]
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

Output

```
Pairs
[('I', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1),
(';', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma', 2),
('Woodhouse', 1), (',', 5), ('handsome', 1), ('clever', 1), (',', 5), ('with', 2),
('a', 1), ('rich', 1), (';', 1), ('life', 1), (';', 1), ('comfortable', 1),
('home', 1), ('and', 3), ('happy', 1), ('disposition', 1), (',', 5),
('seemed', 1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2),
('best', 1), ('blessings', 1), ('of', 2), ('existence', 1), (',', 5),
(';', 1), ('and', 3), ('had', 1), ('lived', 1), ('nearly', 1), ('twenty-one', 1),
('years', 1), ('in', 1), ('the', 2), ('world', 1), ('with', 2)]
```

Experiment 15

Data Collection and Initial Exploration.

Program:

```
import pandas as pd
diabetes_df=pd.read_csv("diabetes.csv")
print(diabetes_df.head())
print(diabetes_df.info())
print(diabetes_df.describe())
```

Output

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness              768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None

       Pregnancies     Glucose  BloodPressure  SkinThickness      Insulin
count   768.000000  768.000000     768.000000     768.000000   768.000000
mean      3.845052  120.894531      69.105469      20.536458    79.799479
std       3.369578   31.972618      19.355807      15.952218   115.244002
min       0.000000    0.000000       0.000000       0.000000     0.000000
25%       1.000000   99.000000      62.000000       0.000000     0.000000
50%       3.000000  117.000000      72.000000      23.000000    30.500000
75%       6.000000  140.250000      80.000000      32.000000   127.250000
max      17.000000  199.000000     122.000000      99.000000   846.000000
■          BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000              768.000000  768.000000  768.000000
mean    31.992578                0.471876   33.240885    0.348958
std      7.884160                0.331329   11.760232    0.476951
min      0.000000                0.078000   21.000000    0.000000
25%     27.300000                0.243750   24.000000    0.000000
50%     32.000000                0.372500   29.000000    0.000000
75%     36.600000                0.626250   41.000000    1.000000
max     67.100000                2.420000   81.000000    1.000000
```

Experiment 16

Time Series Analysis of Website Traffic.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

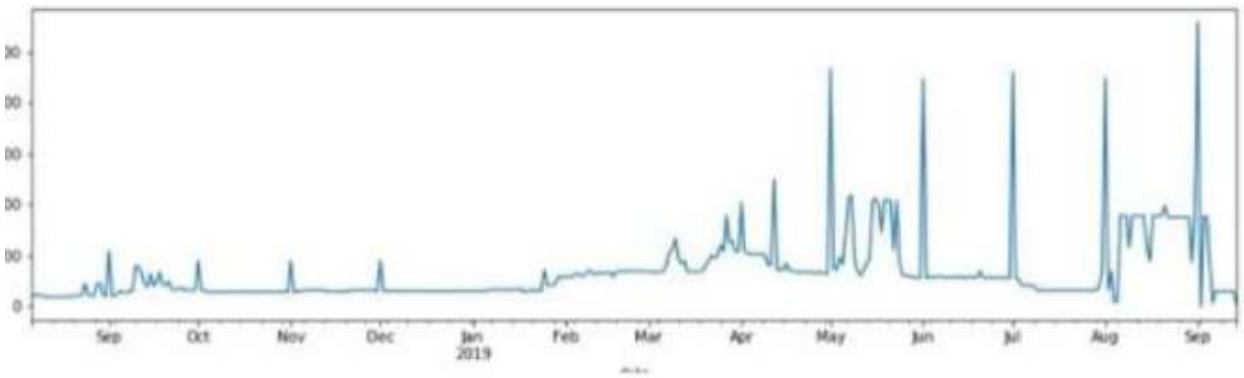
data = pd.read_csv("website_traffic.csv", parse_dates=["Date"], index_col="Date")

decomposition = seasonal_decompose(data["Page_Views"], model="additive")
decomposition.plot()
plt.show()

data["Page_Views_MA"] = data["Page_Views"].rolling(window=30).mean()
data[["Page_Views", "Page_Views_MA"]].plot()
plt.title("Page Views with Moving Average")
plt.show()

data["Page_Views_Z"] = (data["Page_Views"] - data["Page_Views"].mean()) / data["Page_Views"].std()
anomalies = data[data["Page_Views_Z"].abs() > 3]
plt.plot(data.index, data["Page_Views"], label="Page Views")
plt.scatter(anomalies.index, anomalies["Page_Views"], color="red", label="Anomalies")
plt.legend()
plt.show()
```

Output



Experiment 17

Random Sampling and Sampling Distribution.

Program:

```
import numpy as np
import matplotlib.pyplot as plt

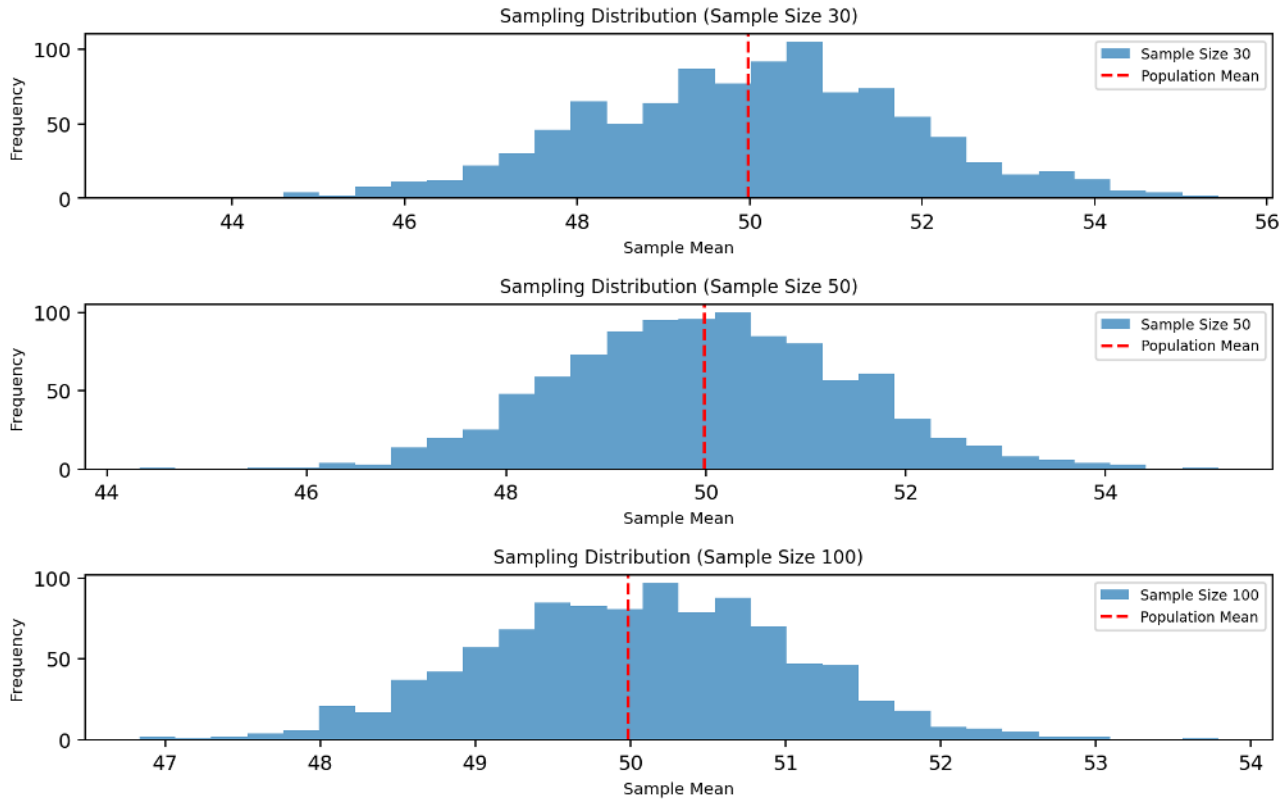
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)

sample_sizes = [30, 50, 100]
num_samples = 1000
sample_means = {}

for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

plt.figure(figsize=(12, 8))
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7,
             label=f"Sample Size {size}")
    plt.axvline(np.mean(population), color="red",
                linestyle="dashed", linewidth=1.5,
                label="Population Mean")
    plt.title(f"Sampling Distribution (Sample Size {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()
plt.tight_layout()
plt.show()
```

Output



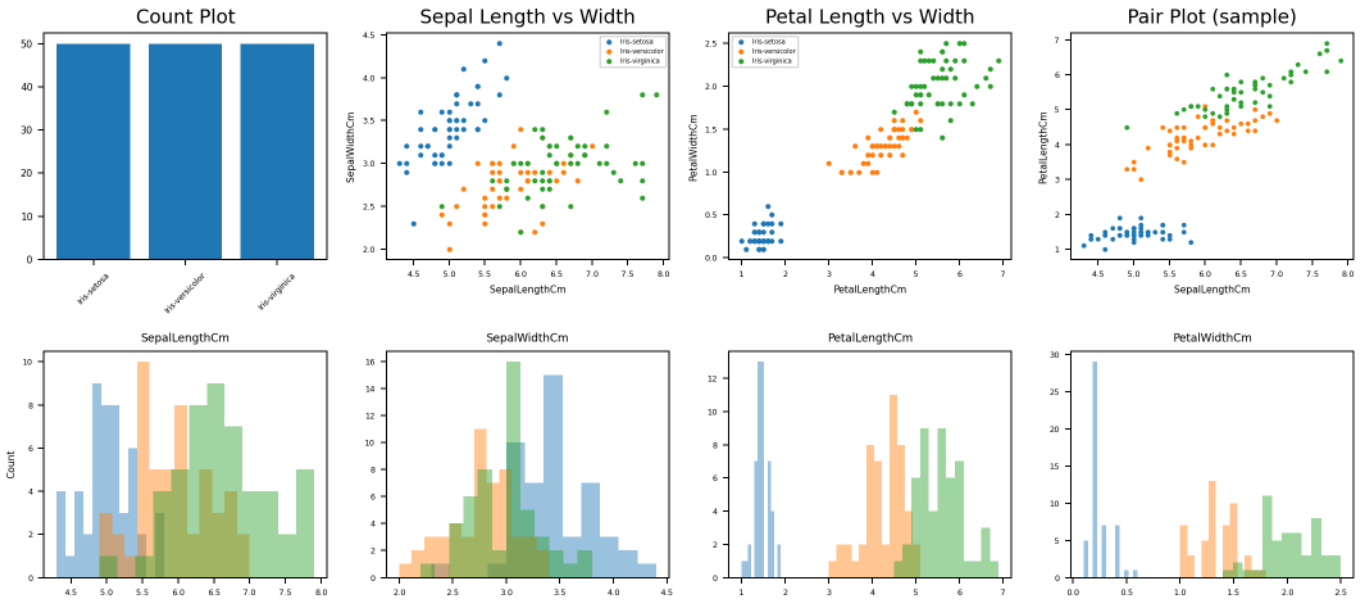
Experiment 18

Matplotlib Library Use Cases in Data Science through Visualization.

Program:

```
import pandas as pd, numpy as np, seaborn as sns, matplotlib.pyplot as plt
%matplotlib inline
data=pd.read_csv("/content/Iris_Dataset.csv")
data.info(); data.describe(); data.value_counts("variety")
sns.countplot(x="variety",data=data); plt.show()
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([dummies,data.iloc[:,[0,1,2,3]]],axis=1)
FinalDataset.head()
sns.scatterplot(x="SepalLengthCm",y="SepalWidthCm",hue="variety",data=data)
sns.scatterplot(x="PetalLengthCm",y="PetalWidthCm",hue="variety",data=data)
sns.pairplot(data,hue="variety",height=3); plt.show()
for col in ["PetalLengthCm","PetalWidthCm","SepalLengthCm","SepalWidthCm"]:
    sns.FacetGrid(data,hue="variety",height=5).map(sns.histplot,col).add_legend()
    plt.show()
```

Output



Experiment 19

Array Functions in Data Science.

Program:

```
import numpy as np
array=np.random.randint(1,100,9)
array
np.sqrt(array)
array.ndim

new_array=array.reshape(3,3)
new_array
new_array.ndim
new_array.ravel()

newm=new_array.reshape(3,3)
newm
newm[2,1:3]
newm[1:2,1:3]
new_array[0:3,0:0]
new_array[0:2,0:1]
new_array[0:3,0:1]
new_array[1:3]
```

Output

```
array([83, 25, 19, 47, 62, 15, 96, 39, 51])

array([9.11043358, 5.        , 4.35889894, 6.8556546 ,
       7.87400787, 3.87298335, 9.79795897, 6.244998  ,
       7.14142843])

1

array([[83, 25, 19],
       [47, 62, 15],
       [96, 39, 51]])

2

array([83, 25, 19, 47, 62, 15, 96, 39, 51])

array([39, 51])
array([[62, 15]])
array([], shape=(3, 0), dtype=int64)
array([[83],
       [47]])
array([[83],
       [47],
       [96]])
array([[47, 62, 15],
       [96, 39, 51]])
```

Experiment 20

Pandas Library Use Cases in Data Science.

Program:

```
import numpy as np
import pandas as pd

list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
df.columns=['Empd','Name','Salary']
df
df.info()

df=pd.read_csv("/content/50_Startups.csv")
df.info()
df.head()
df.tail()

df=pd.read_csv("/content/employee.csv")
df.head()
df.tail()
df.info()

df.salary
type(df.salary)
df.salary.mean()
df.salary.median()
df.salary.mode()
df.salary.var()
df.salary.std()
df.describe()
df.describe(include='all')

empCol=df.columns
empCol
emparray=df.values
employee_DF=pd.DataFrame(emparray,columns=empCol)
employee_DF
```

Output

```
Employee DataFrame output includes:
   Empd / Name / Salary
   1 Smith 50000
   2 Jones 60000

Employee salary statistics:
   mean     5714.285714285715
   median   6000.0
   mode     5000, 6000
   var      2571428.5714285714
   std      1603.5674514745463

Final employee_DF:
   1 SREE VARSSINI K S       5000
   2 SREEMATHI B             6000
   3 SREYA G                 7000
   4 SREYASKARI MULLAPUDI    5000
   5 SRI AKASH U G           8000
   6 SRI HARSHAVARDHANAN R   3000
   7 SRI HARSHAVARDHANAN R   6000
```